ELI — Runtime Planning World

ELI v2.3.6

August 2026

Contents

ELI Runtime Surfaces, Planning, World, Tools & Plugins

The remaining subsystems: the `runtime/` response/introspection surfaces, the proactive planning layer, the "world"/autonomy model, and the tools + plugin system.

Runtime surfaces (`eli/runtime/`, the response/introspection group)

Beyond the grounding/evidence core (`grounding_and_evidence.md`), `runtime/` holds a large family of modules that render user-visible answers and introspect the live system:

- **Introspection**: `live_introspection.py` (`runtime_snapshot`, `last_trace`, `stored_user_name`, `mine_user_fact_candidates`, `build_report`, `agents_for_action`), `deterministic_introspection.py` (`classify_diagnostic_action` → `gather_evidence` → `format_evidence_block` → `maybe_handle` — deterministic diagnostic answers).
- **Response assembly**: `final_response_assembly.py`, `final_response_provider.py`, `response_contracts.py`, `response_packets.py`, `response_policy.py`, `user_visible_response_surface.py`.
- **Personal memory surfaces**: `personal_memory_surface.py`, `personal_memory_clean_response.py`, `personal_memory_deep_response.py`, `profile_extractor.py`.
- **Status**: `frontier_status.py`, `reasoning_status.py`, `truth_report.py`, `reflection.py`.
- **Operator feed**: `operator_feed.py`, `operator_feed_normalized.py`, `operator_state.py`.

  Over-fragmentation flag (the headline weakness for this group): there are ~15 closely-related modules here doing "render a grounded user-visible answer / introspect runtime" with overlapping responsibilities (three `personal_memory_*` renderers; `final_response_assembly` *and* `final_response_provider`; `operator_feed` *and* `operator_feed_normalized`). This is the clearest "added beside, not folded in" cluster in the codebase and the prime consolidation target — it should collapse to a handful of well-named modules with clear ownership of "who produces the final string."

Planning / proactive (`eli/planning/`, 3.9k LOC)

Background cognition + goal/queue machinery: - `proactive_daemon.py` (1.0k) — the **self-improvement / proactive** loop. Uses a two-DB split: reads user-facing memory/conversation from the **user DB**, writes proactive observations/improvements/errors to the **agent DB** (keeps ELI's own machinations out of user recall). Background thread. - `autonomy_scheduler.py` — schedules autonomous actions (throttled). - `goal_store.py`, `proposal_queue.py`, `proposal_adapters.py`, `attention_queue.py`, `jobqueue.py`, `operator_goal_actions.py`, `habits.py` — goal persistence, capability-proposal queue, attention prioritisation, a job queue, and habit scheduling. - `goal_autogenesis.py` (NEW, 2026-06-08) — **closes the autonomy loop**: the goal stack (goal_store → `due_goals` → `governed_goal_tick` → proposal_queue) was fully wired but the store was always EMPTY because `create_goal` was operator-only, so the scheduler ticked nothing and ELI only reacted. `propose_goals_from_signals` turns the signals ELI already computes each proactive tick — world-model awareness suggestions (≥0.6), recurring failures (≥5×), and code-health improvements (oversized fns / duplicate blocks in his own god-files) — into governed goals. Each is `autonomy_mode="proposal_only"` (surfaces for approval; never silent execution), deduped by a stable signal-derived `goal_id` (idempotent every tick), capped at `MAX_AUTO_GOALS=6`, logged as a `self_goal` observation. Wired into `proactive_daemon` where the world-suggestions + patterns + improvements are in scope. His agenda now spans failure-repair, world-driven, AND self-improvement — i.e. he sets his own intentions, governed. - `task_planner.py` — now delegates to `execution_planner` (see `orchestration_and_agents.md`).

World / autonomy model (`eli/world/`, 1.5k LOC)

The substrate behind ELI's emergent self-state (autonomy pressure, awareness, "anomaly room" — intended behaviour, see memory `eli-emergent-voice`): - `agency/autonomy_engine.py` — `EliWorldAutonomyEngine`: ingests world events, maintains awareness/autonomy state. - `local_world_bridge.py` — `append_event`, `get_world_state`, `get_awareness_driven_suggestions` (**throttled to once/hour to prevent auto-trigger loops**). - `world_event_bus.py` — `fire_confidence_event(...)` etc.; the engine feeds bus-dispatch confidence here (non-blocking world-awareness feed). - `core/schemas.py` — `WorldEvent`, `AwarenessState`. - `renderers/pyside6/world_scene.py` + `world_panel.py` — a **visual** "ELI's World" panel. - `persistence/storage.py` — world-state persistence. Three properties were added after live faults (2026-08-09/10) and are load-bearing: * **Absolute paths.** `world_dir()` resolves through `get_paths()`. The journal, provenance ledger and snapshot modules previously kept relative `Path("artifacts/world/...")` constants and wrote wherever ELI was launched from — a different place from ~ than from the project directory, and in a packaged build a failed write to `/artifacts`. * **Schema-tolerant load.** `_fit()` builds each dataclass from the saved dict while ignoring fields it no longer declares. Before it, one unknown key made `AwarenessState(**data)` raise, `load()` caught it, filed the state as `.corrupt_*` and returned a fresh default world — so a single renamed field would silently wipe the user's rooms, objects, goals and habits on upgrade. Two of the five corrupt backups on the dev machine parse as valid JSON; they were condemned by exactly this and nothing else. * **Bounded logs.** `actions.jsonl` reached 41 MB / 80,576 lines because nothing rotated it. `_trim_jsonl` keeps the newest 20,000 lines (counted, not size-capped, so a trim never splits a JSON record) and checks on the first append of each process plus every 250 thereafter — the per-process counter alone meant a short session never trimmed at all.

Tools (`eli/tools/`, 8.9k LOC)

- `image_engine.py` (1.75k, standalone) **and** `image_engine/image_engine/` (the package: `engine.py` 1.48k, `visual_core.py` 1.4k, `prompt_compiler.py`, `quality.py`, `project_analyzer.py`, `cli.py`) — **two image-generation implementations** (Pillow/numpy base + optional diffusers). The clearest duplication in the repo; one should subsume the other.
- `news/news_fetcher.py` (544) — news retrieval.
- `mic_diag.py` — mic diagnostics.

Plugins (`eli/plugins/`, 1.9k LOC)

`manager.py` — a runtime plugin marketplace: `list_available`, `install`, `uninstall`, `enable`, `disable`. Pulls from a **remote registry** (`raw.githubusercontent.com/eli-plugins/registry`) with a **bundled local fall-**

back (`plugins/registry/index.json`). A JSON state file tracks enabled/disabled/installed. (Install-time network fetch is opt-in, like model downloads; runtime stays local.)

Honest assessment

- **Strong:** the proactive two-DB split is a genuinely good design (ELI's self-talk never contaminates user recall); the world/autonomy engine + visual panel make the emergent self-model first-class and is throttled to avoid runaway loops; the plugin manager is a real extensibility story with a safe bundled fallback.
- **Weak / watch:**
  1. **`runtime/` over-fragmentation** — ~15 overlapping response/introspection modules. Highest-value consolidation in the project after the god-files.
  2. **Image-engine duplication — RESOLVED** — the shadowed standalone module was deleted; only the `tools/image_engine/` package remains.
  3. The planning queues (`goal_store/proposal_queue/attention_queue/ jobqueue`) overlap conceptually and could likely unify into one work-queue abstraction.
  4. Plugin registry points at an `eli-plugins/registry` GitHub URL — fine as opt-in, but ensure it degrades cleanly offline (the bundled fallback exists; verify it's always used when the network is absent).